



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CIÊNCIA DA COMPUTAÇÃO

Rodrigo Schwartz e Vinicius Henrique Ribeiro

Trabalho I - Transferência de Arquivos Distribuída

Florianópolis
2026

O presente relatório tem como objetivo explicar o funcionamento da solução desenvolvida para o primeiro trabalho da disciplina INE 5418 - Computação Distribuída, do curso de Ciências da Computação da Universidade Federal de Santa Catarina.

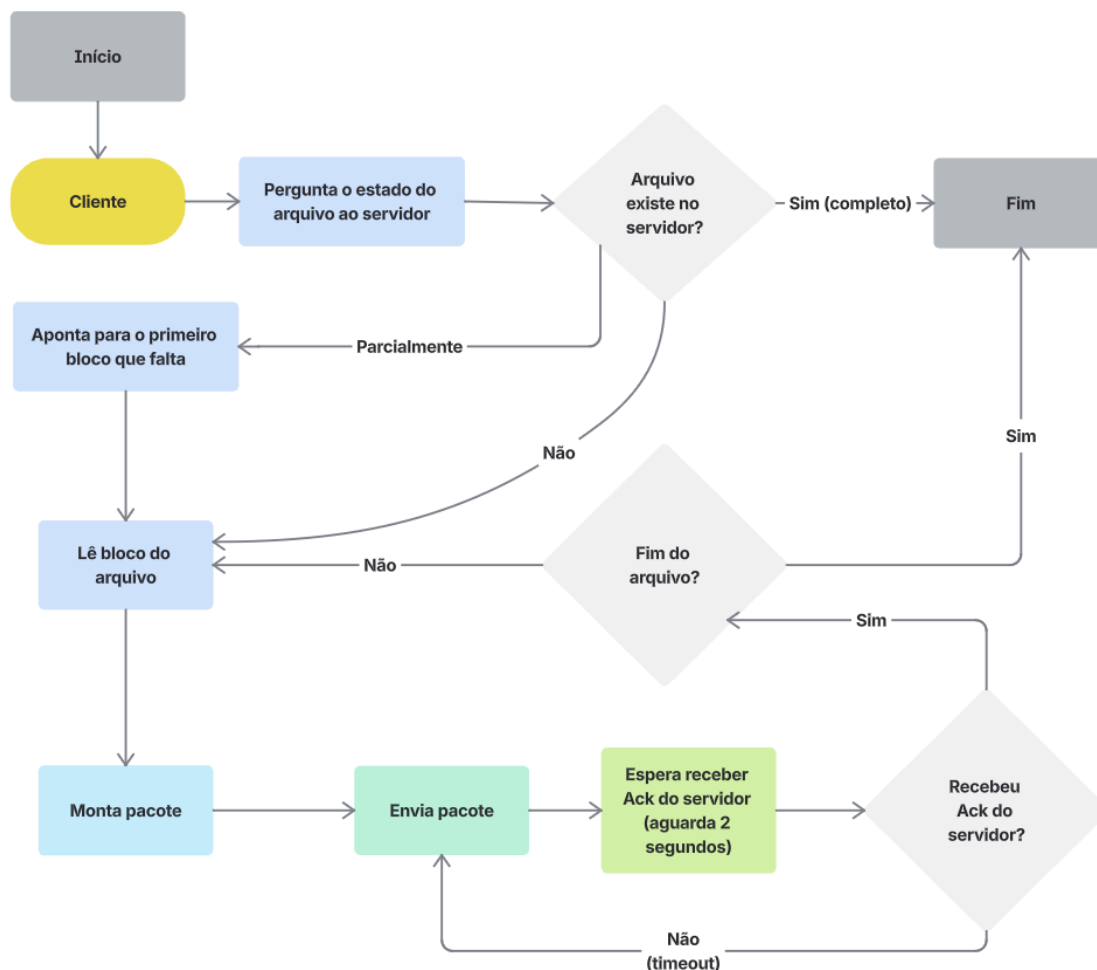
O sistema desenvolvido consiste em uma aplicação para realizar o envio de arquivos diversos a um servidor, que deve armazená-los. A arquitetura da solução desenvolvida baseia-se no modelo cliente-servidor, foi programada em linguagem C e utiliza Sockets Berkeley e o protocolo de rede UDP para estabelecer a conexão entre o cliente e o servidor, e realizar a transferência dos dados.

A utilização do protocolo UDP faz com que não haja garantia de entrega de pacotes, não haja garantia na ordem de envio e que haja a possibilidade de um arquivo ficar incompleto, sem ser possível retransmiti-lo de onde se havia parado. Além disso, a possibilidade de concorrência entre diferentes clientes realizando upload de arquivos de modo simultâneo ao servidor também é uma questão a ser destacada. Por conta disso, a solução desenvolvida traz estratégias visando resolver estes problemas, para que a transferência de arquivos garanta entrega de pacotes em ordem e permita realizar a retransmissão de um arquivo de onde se havia parado, caso o servidor ou o cliente tenha algum problema.

Para garantir a entrega dos pacotes, utilizou-se o protocolo Stop-and-Wait (Pare-e-Espere). Nele, o cliente só envia um novo pacote após receber do servidor a confirmação (ACK) de que o pacote anterior foi recebido. Caso essa confirmação não chegue em até 2 segundos, seja porque o pacote ou o ACK se perdeu na rede, o cliente retransmite o mesmo pacote. Se o servidor receber um pacote que já possui, ele o descarta, mas envia o ACK novamente, evitando que os dados sejam regravados.

Para lidar com a falha do processo cliente, o cliente, antes de enviar o arquivo, consulta o servidor sobre o estado dele. O servidor pode responder que o arquivo não existe e, nesse caso, o cliente envia o arquivo completo. Pode responder que o arquivo existe, mas está incompleto, informando até qual byte já o possui, e o cliente envia somente a parte que falta. Por fim, pode responder que o arquivo já está completo, e o cliente apenas encerra sua execução. Dessa forma, se o cliente for interrompido no meio de uma transferência, basta executá-lo novamente para que ela seja retomada do ponto em que parou.

O fluxo de execução do cliente para lidar com a garantia de entrega de pacotes e com a retransmissão pode ser visualizado na figura abaixo:

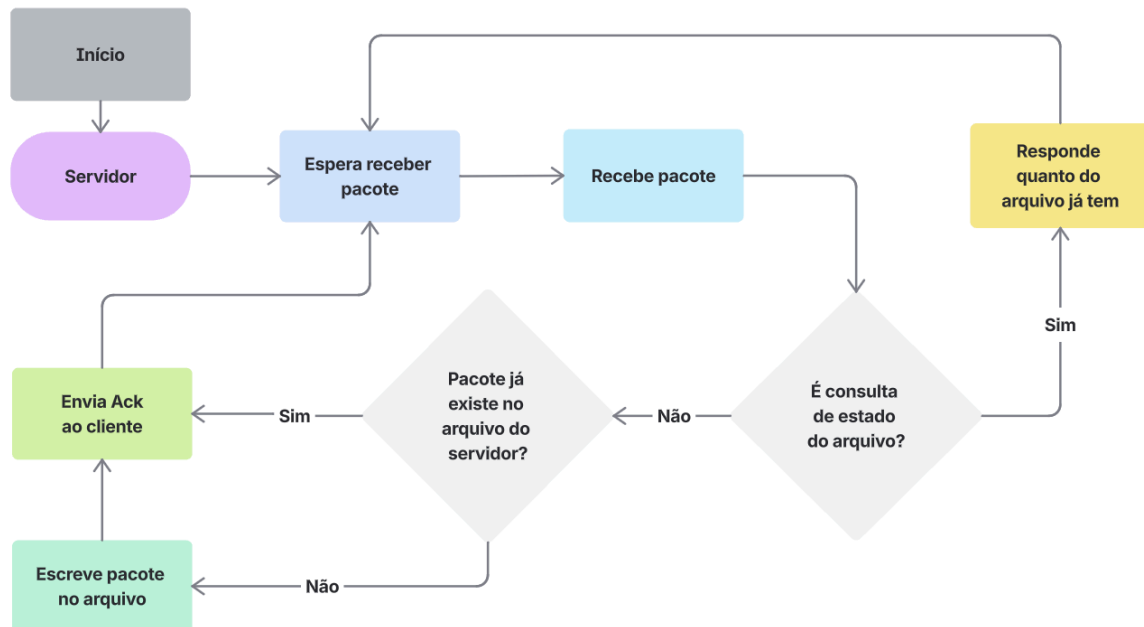


Fluxo de execução do cliente

Em um cenário de falha do cliente, onde um arquivo foi enviado parcialmente, o cliente, ao ser executado novamente, consulta o servidor sobre a situação do arquivo. O servidor responde que possui o arquivo parcialmente e junto com essa informação, informa quantos bytes já recebeu. De posse dessa informação, o cliente consegue então apontar para esse byte, e ler a partir dele. Para cada bloco faltante, o cliente monta o pacote de dados, envia ao servidor, e fica esperando uma resposta (ACK) do mesmo. Se o cliente receber a resposta do servidor, confirmando que recebeu o pacote de dados, o cliente lê então o próximo bloco. Se o cliente não receber a resposta em até 2 segundos, reenvia o mesmo pacote, até receber a confirmação de recebimento. Esse fluxo se repete até que todo o arquivo tenha sido enviado e confirmado pelo servidor. Dessa forma, garante-se a

entrega dos pacotes em ordem e sem duplicação, mesmo diante de perdas na rede ou de uma interrupção do cliente.

O fluxo de execução do servidor pode ser visualizado na figura abaixo:



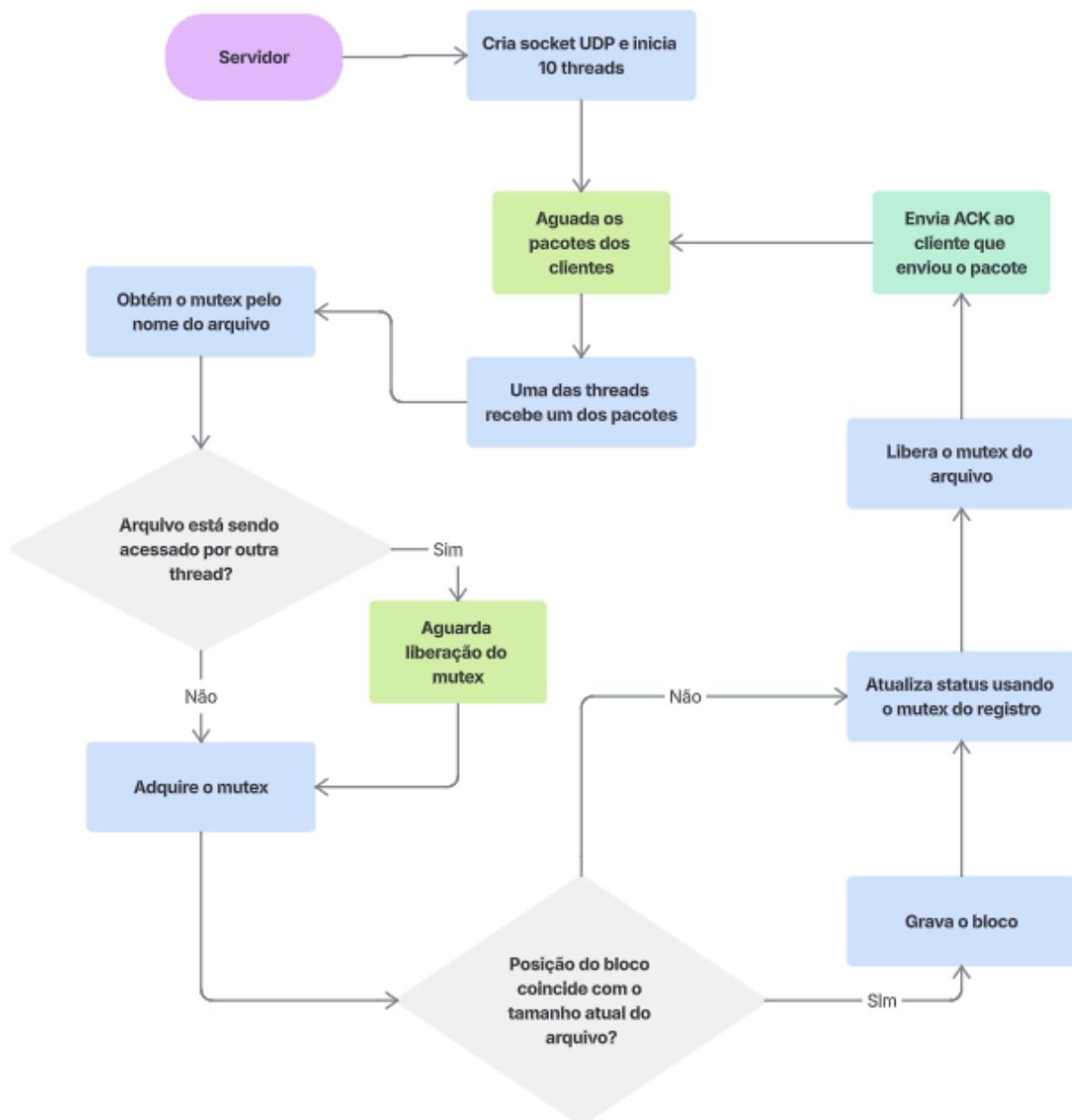
Fluxo de execução do servidor

Em uma transmissão de arquivos por clientes concorrentes a nossa implementação utiliza um conjunto fixo de 10 *threads* no servidor de *upload*. Todas utilizam o mesmo *socket* UDP e aguardam datagramas por meio de *recvfrom()*. O endereço retornado por *recvfrom()* permite enviar a resposta ao cliente correspondente.

Para controlar o acesso aos arquivos, o servidor mantém uma tabela *hash* que associa cada nome de arquivo a um *mutex*. Antes de fazer a consulta ou modificar o arquivo, a *thread* adquire o *mutex*, segurando ele por todo o processamento do datagrama, e não durante toda a transferência do arquivo. Isso permite paralelismo de escrita em arquivos diferentes, mantendo a escrita em um mesmo arquivo serial.

Dentro da área protegida, o servidor verifica se o datagrama já foi escrito por outra *thread* usando o tamanho do arquivo e o *current_seek* do cabeçalho do datagrama, que diz o início de onde foi lido os dados do arquivo do cliente. Este caso pode acontecer se um cliente não receber um *ACK* e reenviar o mesmo datagrama, ocasionando dois datagramas iguais na fila do *socket* e um possível processamento do mesmo bloco de dados em duas

threads diferentes. A tabela hash também possui um mutex próprio para controlar as operações internas, evitando que um mutex seja excluído enquanto outra *thread* esteja o utilizando. O arquivo de status possui um mutex global, *status_file_lock*, que serializa as operações feitas pelo servidor de *upload*. O fluxo do pool de threads pode ser visualizado abaixo:



Fluxo de execução do pool de threads do servidor

A utilização do protocolo TCP facilitaria a solução do trabalho devido às vantagens que o protocolo fornece no quesito da garantia de entrega de pacotes e na retransmissão em caso de perdas. Enquanto no protocolo UDP não há garantia de entrega confiável, em ordem e retransmissão, no protocolo TCP esses problemas já são tratados internamente.

Isso permite com que o desenvolvedor da aplicação não precise lidar com essas questões. Como não temos essas garantias utilizando o UDP, boa parte da solução do trabalho pautou-se no desenvolvimento de uma camada de software para aprimorar este protocolo.

Repositório do projeto: <https://github.com/R0drigoSchwartz/distributed-file-transfer>